BINARY TREE & BST MASTERY

A step-by-step guide from beginner to advanced LeetCode problem solving

Learn the fundamentals, understand recursion, master DFS and BFS, recognize patterns, and build solutions systematically — with Java code and dry runs.

LEVEL 1	Tree basics, node references, DFS, BFS, recursion
LEVEL 2	Easy problems, paths, backtracking, BST fundamentals
LEVEL 3	Balanced trees, diameter, maximum path sum, LCA, views
LEVEL 4	Construction, distance K, serialization, advanced patterns
LEVEL 5	LeetCode learning path, complexity, edge-case checklist

How to study: Do not memorize the code first. Read the explanation, trace the example, explain what the helper function means, then write the code without looking.

A step-by-step guide from beginner to advanced LeetCode problem solving

This guide is designed to teach you how to *think through* tree problems, not just memorize solutions. Work through the chapters in order. Every pattern builds on an earlier one.

Part 1 — Build the Foundation

1. What is a tree?

A tree is a non-linear data structure made of nodes connected by edges. Unlike an array, a tree does not store elements in one straight sequence. A node can connect to child nodes, creating branches.

A **binary tree** is a tree where each node can have at most two children: - `left`: reference to the left child - `right`: reference to the right child

Example:

```
        1
      /   \
     2     3
    / \     \
   4   5     6
```

In this tree: - `1` is the root. - `2` and `3` are children of `1`. - `4`, `5`, and `6` are leaf nodes. - The subtree rooted at `2` contains `2`, `4`, and `5`.

2. The Java node structure

Most LeetCode tree questions provide this class. You usually do not need to write it yourself.

Understand the references: - `root.left` gives the left child. - `root.right` gives the right child. - `root.val` gives the current node's value. - `root == null` means there is no node at that position.

Very important: `root` is a reference to a node, not the node's value. Two different nodes can have the same value.

3. Tree vocabulary you must know

Term	Meaning
Root	First/top node
Parent	A node directly above another node
Child	A node directly below another node
Leaf	Node with no children
Subtree	A node and all descendants below it
Depth	Number of edges from root to a node
Height	Longest downward path from a node to a leaf
Level	Group of nodes at the same depth
Ancestor	Any node above a given node
Descendant	Any node below a given node
Diameter	Longest path between any two nodes

Height conventions vary: some questions count **edges**, others count **nodes**. Read the statement carefully. In this guide, the `height()` method returns the number of nodes on the longest root-to-leaf path, so an empty tree has height `0` and a leaf has height `1`.

4. How to read a tree input

LeetCode often displays a tree as level-order values:

```
[1, 2, 3, 4, 5, null, 6]
```

It represents:

```
        1
      /   \
     2     3
    / \     \
   4   5     6
```

`null` means that a child is missing. It is not a node whose value is zero.

5. The first four questions to ask

Before coding, identify:

1. **What is given?** A binary tree, a BST, two trees, or a target node?
2. **What is required?** A value, boolean, list, path, count, or modified tree?
3. **What is the relationship?** Parent-to-child, child-to-parent, or level-by-level?
4. **What does one helper call mean?** Define this before writing recursion.

Part 2 — Learn the Two Ways to Move Through a Tree

6. DFS and BFS: the big picture

There are two fundamental traversal strategies.

DFS — Depth First Search

Go down one branch before exploring another. Commonly implemented using recursion or a stack.

Use DFS when: - You need subtree information. - You need paths from root to leaf. - You need to compare or validate whole subtrees. - You need height, diameter, or maximum path information.

BFS — Breadth First Search

Visit all nodes at one level before moving to the next. Implemented using a queue.

Use BFS when: - The question says level, nearest, minimum number of edges, or shortest distance in an unweighted tree. - You need left/right views or level summaries. - You need to spread outward from a target node.

7. DFS traversal orders

There are three standard DFS orders. `N` means current Node, `L` means Left, and `R` means Right.

Traversal	Order	When the node is processed
Preorder	N → L → R	Before its children
Inorder	L → N → R	Between its children
Postorder	L → R → N	After its children

Use this tree for all examples:

```
      1
     / \
    2   3
   / \
  4   5
```

1. Preorder: `[1, 2, 4, 5, 3]`
1. Inorder: `[4, 2, 5, 1, 3]`
1. Postorder: `[4, 5, 2, 3, 1]`

7.1 Preorder — Node, Left, Right

Think: **process me first, then ask my children to process themselves.**

7.2 Inorder — Left, Node, Right

Think: **finish left side, process me, then finish right side.**

7.3 Postorder — Left, Right, Node

Think: **children first, process me last.**

7.4 The universal recursive DFS template

This template is the foundation of most recursive tree solutions.

Part 3 — Recursion, Explained Properly

8. How recursion actually works

Consider:

For this tree:

```
    1
   / \
  2   3
 /
4
```

The calls go downward until they reach `null`. Then answers return upward:

1. `height(null)` returns `0`.
2. Node `4` gets left `0`, right `0`; returns `1`.
3. Node `2` gets left `1`, right `0`; returns `2`.
4. Node `3` gets left `0`, right `0`; returns `1`.
5. Node `1` gets left `2`, right `1`; returns `3`.

The important idea: **a recursive call solves a smaller subtree and gives its result back to the caller.**

9. The three recursion patterns behind most tree questions

Pattern A — Return information from children

Use when the parent needs answers from its left and right subtrees.

Examples: - Height - Number of nodes - Balanced tree - Diameter - Maximum path contribution

Steps: 1. Define what the function returns for one subtree. 2. Write the null base case. 3. Call the function for left and right. 4. Combine both answers with the current node. 5. Return the combined result.

Pattern B — Pass information from parent to child

Use when each node needs information accumulated along the path from the root.

Examples: - Root-to-leaf sum - Path to a target - Current depth - Ancestor constraints

Each call receives its own `int` value. Updating `runningSum` in one call does not mutate the caller's integer.

Pattern C — Global answer + local return

Use when the best answer may be located anywhere, but each node returns only a useful partial result to its parent.

Examples: - Diameter - Maximum path sum - Best path through any node

Why return one branch only? A path returned to the parent cannot split into two directions. A complete path may use both left and right branches at its highest point, but it cannot then continue upward.

10. How to design a recursive helper — write this before code

For every recursive tree problem, fill in these sentences:

1. `dfs(node)` means: _____
1. If `node == null`, return: _____
1. Left call calculates: _____
1. Right call calculates: _____
1. Current node combines them by: _____
1. The function returns: _____
1. The final answer is obtained from: root return / global variable / list / boolean.

If you cannot fill these in, you are not ready to code yet.

Part 4 — Iterative DFS and BFS

11. Why use a stack or queue?

Recursion uses the program's call stack. Iterative traversal uses a data structure that you control.

1. **Stack:** Last In, First Out (LIFO). Used for iterative DFS.
1. **Queue:** First In, First Out (FIFO). Used for BFS.

In Java, prefer `Deque<TreeNode>` with `ArrayDeque` for stack operations, and `Queue<TreeNode>` with `ArrayDeque` for queue operations. `ArrayDeque` does not allow `null` elements, which is fine because we only enqueue real nodes.

12. Iterative preorder — one stack

Push right before left, because the stack pops the most recently pushed node.

13. Iterative inorder — one stack

Keep going left and save ancestors. When there is no left child, process the most recent ancestor and move right.

14. Iterative postorder — two stacks

The first stack visits nodes in an order that the second stack reverses into Left → Right → Node.

Memory rule: postorder LRN is the reverse of NRL (Node → Right → Left), not the reverse of ordinary preorder NLR.

15. Level order — the most important BFS template

Use a queue. The key trick is to capture the queue size before processing the current level.

Dry run the queue

For:

```
    1
   / \
  2   3
 / \
4   5
```

Stage	Queue	Current level output
Start	`[1]`	—
Process 1	`[2, 3]`	`[1]`
Process 2 and 3	`[4, 5]`	`[2, 3]`
Process 4 and 5	`[]`	`[4, 5]`

Answer: `[[1], [2, 3], [4, 5]]`.

Why save `size`? Children added during this level belong to the next level. The fixed `size` prevents processing them too early.

BFS variations

1. Zigzag: reverse alternate level lists, or fill indexes in reverse.
1. Right view: take the last node of each level.
1. Left view: take the first node of each level.
1. Level averages: sum values and divide by `size`.
1. Minimum depth: first leaf encountered in BFS.
1. Bottom-up level order: reverse the list of levels.

Part 5 — Start Solving Easy Tree Problems

16. Maximum depth of a binary tree

Question idea: How many nodes are on the longest root-to-leaf path?

Pattern: return information from children.

Step-by-step: 1. Empty tree has depth `0`. 2. Ask the left subtree for its depth. 3. Ask the right subtree for its depth. 4. Choose the larger depth. 5. Add one for the current node.

Time: `O(n)`. Recursion space: `O(h)`.

17. Same tree

Question idea: Do two trees have the same structure and the same values?

Pattern: compare two nodes together.

The null check handles three cases: - Both null: same empty position. - Only one null: different structure. - Both non-null: compare values and children.

18. Invert a binary tree

Question idea: Swap left and right at every node.

Process the current node's swap, then recursively invert its children. Time: `O(n)`.

19. Symmetric tree

Question idea: Is the tree a mirror of itself?

Compare the left and right sides in opposite directions.

Do not compare left with left and right with right. Mirror comparison crosses the children.

20. Search in a normal binary tree

A normal binary tree has no sorted ordering. In the worst case, inspect every node.

Time: `O(n)`.

21. Count nodes

The `1` counts the current node; the recursive calls count the two subtrees.

Part 6 — Paths and Backtracking

22. Understand the different meanings of “path”

Phrase in question	Meaning
Root to leaf	Must start at root and end at a leaf
Root to node	Starts at root, ends at a target
Downward path	Moves from a node to descendants
Any to any	Can start and end anywhere
Path through a node	May use both child branches at that node

A leaf means **both** children are null. A node with only one child is not a leaf.

23. Root-to-leaf path sum

Question idea: Does at least one root-to-leaf path add up to the target?

Step-by-step: 1. If the current position is null, no path exists. 2. If it is a leaf, check whether its value completes the remaining sum. 3. Otherwise subtract the current value. 4. Search either child.

Do not return true just because an intermediate node reaches the target. The question requires a leaf.

24. Find all root-to-leaf paths — backtracking

Backtracking has three actions: 1. Choose: add the current node. 2. Explore: recursively visit children. 3. Undo: remove the current node before returning.

Why copy the list? The same mutable `path` list is reused. Without `new ArrayList<>(path)`, all saved answers would refer to a list that keeps changing.

25. Find a path from root to a target

This version identifies a target by value. If duplicate values are possible and the problem identifies a particular node, compare node references instead.

Part 7 — Learn the Binary Search Tree

26. What makes a BST different?

A Binary Search Tree follows an ordering rule. Under the common distinct-value definition:

1. Every value in the left subtree is smaller than the node.
1. Every value in the right subtree is larger than the node.
1. Both subtrees must also obey the same rule.

Example:



A BST is still a binary tree, but it gives you an ordering advantage.

27. Search in a BST

At every node, eliminate one entire side. Average/balanced time: `O(log n)`. Worst-case time in a skewed BST: `O(n)`.

28. Validate a BST — use a range, not only parent comparisons

A common wrong solution checks only whether the left child is smaller and the right child is larger. That misses violations deeper in a subtree.

Correct idea: every node inherits a valid range from its ancestors.

Why `long`? So `Integer.MIN_VALUE` and `Integer.MAX_VALUE` are both valid node values.

This version assumes duplicates are not allowed. If duplicates are allowed, the problem must define which side can contain equal values, and the inequalities must be adjusted.

29. Kth smallest in a BST

Inorder traversal of a BST visits values in sorted order. Therefore, the kth visited node is the kth smallest.

30. Lowest Common Ancestor in a BST

The first node where the two targets split to different sides (or one target equals the current node) is the LCA.

Part 8 — Intermediate Tree Properties

31. Minimum depth — watch out for a missing child

Minimum depth is the number of nodes on the shortest root-to-leaf path.

Why not always use `1 + min(left, right)`? If one child is missing, its depth is zero, but that null position is not a leaf path. You must follow the existing child.

32. Balanced binary tree

A tree is height-balanced if at every node, the left and right subtree heights differ by no more than one.

Naive approach: calculate height separately at every node. This can take $O(n^2)$ on a skewed tree.

Optimized approach: calculate height once and use `-1` to signal imbalance.

Time: $O(n)$. Each node is processed once.

33. Diameter of a binary tree

Diameter is the longest path between any two nodes. The path does not have to pass through the root.

At a node: - Left height tells how far the path can extend left. - Right height tells how far it can extend right. - A path through this node has `left + right` edges when height is measured in nodes.

Reset `diameter = 0` for each new test case. This implementation returns diameter in **edges**.

34. Maximum path sum — advanced recursion pattern

A path can begin and end anywhere, but it cannot branch. At each node: - The global candidate can use left + node + right. - The value returned to the parent can use only one side.

Why use `Integer.MIN_VALUE`? The tree may contain only negative values. Initializing the answer to zero would incorrectly allow an empty path if the problem requires at least one node.

Part 9 — Tree Relationships and Distance

35. Lowest Common Ancestor in a normal binary tree

For a normal binary tree, there is no sorted order to guide the search.

This standard version assumes both nodes exist. If the problem does not guarantee that, also track whether both targets were found.

36. Nodes at distance K / Burning tree

A binary tree normally has child references but no parent reference. To move upward from a target, create a parent map.

Step-by-step: 1. Traverse the tree and record `parent[child] = current`. 2. Start BFS from the target node. 3. For every visited node, consider its left child, right child, and parent. 4. Track visited nodes so you do not travel back and forth. 5. For distance K, stop after K layers. For burning time, continue until no nodes remain.

Use node references as map keys if duplicate values are possible.

Part 10 — Tree Views

37. Left and right side view

Use level order traversal: - Left view: first node processed at each level. - Right view: last node processed at each level.

38. Top and bottom view

Assign a horizontal distance (HD): - Root has HD `0`. - Left child has HD `parentHD - 1`. - Right child has HD `parentHD + 1`.

Use BFS and store values by HD: - Top view: first node seen at each HD. - Bottom view: deepest node at each HD; tie-breaking depends on the problem.

39. Vertical order traversal

Track two coordinates: - Horizontal distance (column). - Depth (row).

Left child: `(column - 1, row + 1)`. Right child: `(column + 1, row + 1)`.

Group by column. If multiple nodes share a position, follow the exact sorting rules in the question.

40. Boundary traversal

A common anti-clockwise boundary is: 1. Root. 2. Left boundary, excluding leaves. 3. All leaves from left to right. 4. Right boundary, excluding leaves, in reverse order.

Be careful not to add leaf nodes twice.

Part 11 — Constructing Trees

41. Build from preorder + inorder

Key facts: - First preorder element is the root. - Find that value in inorder. - Values left of it belong to the left subtree. - Values right of it belong to the right subtree.

Use a map from value to inorder index for efficient lookup. This common approach assumes unique values.

Before calling `build`, populate `index` with each inorder value and its position. The initial call is `build(preorder, 0, n - 1, 0, n - 1)`.

42. Build from inorder + postorder

Key facts: - Last postorder element is the root. - Find it in inorder to split the subtrees. - When consuming postorder backwards, build the **right subtree first**, then the left.

43. Can preorder + postorder uniquely construct any binary tree?

No. In general, preorder and postorder alone do not uniquely determine a binary tree. Extra information, such as the tree being full, may make construction possible.

44. Sorted array to balanced BST

Choose the middle element as root. Recursively build the left half and right half. This keeps the resulting tree height-balanced.

Part 12 — Serialization and Pointer Changes

45. Serialization and deserialization

Serialization turns a tree into a string or sequence. Deserialization rebuilds the same tree.

A common method is preorder plus explicit null markers:

```
1,2,#,#,3,#,#
```

`#` represents a missing child. Without null markers, different tree shapes can produce the same value sequence.

46. Flatten a tree to a linked list

If the required order is preorder: - Each node’s left pointer must become null. - Its right pointer must point to the next node in preorder.

A useful strategy is reverse preorder: 1. Process right subtree. 2. Process left subtree. 3. Link current node to the previously processed node.

Reset `prev = null` before processing a new tree.

Part 13 — How to Recognize the Pattern Quickly

47. Keyword-to-pattern map

If the question says…	Think…
Traverse / print in order	DFS
Level / nearest / minimum steps	BFS
Height / depth	Return from children
Balanced	Postorder height + early failure
Diameter	Left height + right height
Maximum path sum	Global answer + one-branch return
Root-to-leaf sum	Carry remaining sum
All paths	Backtracking
Mirror / symmetric	Compare opposite children
Same tree	Compare matching children
BST / kth smallest	Inorder or range bounds
LCA	BST ordering or general-tree recursion
Distance K / burning	Parent map + BFS
Top / bottom / vertical	Horizontal distance + BFS
Build from traversals	Root position + subtree boundaries
Flatten	Pointer rewiring

48. A five-step problem-solving routine

Step 1 — Identify the output. Is it a number, boolean, list, path, or modified tree?

Step 2 — Identify the movement. DFS, BFS, or BST-directed search?

Step 3 — Define the helper. What exactly does `dfs(node)` return or carry?

Step 4 — Decide where the answer is formed. Returned from root, updated globally, or collected in a list?

Step 5 — Dry run and test. Use a small tree, then test edge cases.

49. A full example of thinking before coding

Problem: Find the diameter of a binary tree.

1. Output: one integer — the longest path in edges.
2. The path may be anywhere, not necessarily through root.
3. At each node, I need left height and right height.
4. Candidate diameter through this node is `left + right`.
5. The parent needs only one branch: `1 + max(left, right)`.
6. Therefore use postorder DFS and a global answer.

This reasoning leads directly to the diameter code in Chapter 33.

Part 14 — Complexity and Edge Cases

50. Complexity reference

Let: - `n` = number of nodes. - `h` = height of the tree. - `w` = maximum width of the tree.

Pattern	Time	Extra space
DFS traversal	O(n)	O(h) recursive stack
Iterative DFS	O(n)	O(h) typical stack; up to O(n)
Level order BFS	O(n)	O(w) queue
Search normal binary tree	O(n)	O(h)
Search balanced BST	O(log n)	O(log n)
Search skewed BST	O(n)	O(n)
Height / count / balanced / diameter	O(n)	O(h)
Same tree / symmetry	O(n)	O(h)
Build from traversals with index map	O(n)	O(n)
Parent map + BFS	O(n)	O(n)

A skewed tree can have height `n`; a balanced tree has height around `log n`.

51. Edge-case checklist

Before submitting, test:

1. Empty tree.
2. One node.
3. Only-left-child chain.
4. Only-right-child chain.
5. Perfect or complete tree.
6. Highly unbalanced tree.
7. Negative values.
8. Duplicate values, if allowed.
9. Target does not exist.
10. Target is the root.
11. All values are negative for maximum path sum.
12. Very deep tree — recursion depth may be a concern.

52. Common mistakes

1. Forgetting the `null` base case.
1. Treating a normal binary tree like a BST.
1. Checking only immediate children when validating a BST.
1. Confusing a missing child with a leaf in minimum-depth problems.
1. Returning a two-branch path to the parent.
1. Forgetting to copy a path before storing it.
1. Forgetting to undo a backtracking choice.
1. Not capturing `queue.size()` before processing a BFS level.
1. Pushing left before right in iterative preorder.
1. Assuming values uniquely identify nodes.
1. Forgetting to reset global or instance variables between test cases.
1. Using `int` for sums that may overflow.

[1] Assuming both LCA targets exist when the question does not guarantee it.

Part 15 — Your LeetCode Learning Path

53. Beginner: learn the shape of a tree

Solve in this order: 1. 144 — Binary Tree Preorder Traversal 2. 94 — Binary Tree Inorder Traversal 3. 145 — Binary Tree Postorder Traversal 4. 102 — Binary Tree Level Order Traversal 5. 104 — Maximum Depth of Binary Tree 6. 100 — Same Tree 7. 226 — Invert Binary Tree 8. 101 — Symmetric Tree 9. 112 — Path Sum 10. 110 — Balanced Binary Tree

Goal: understand null checks, recursive calls, stack, queue, and return values.

54. Intermediate: combine information

1. 111 — Minimum Depth of Binary Tree
2. 222 — Count Complete Tree Nodes
3. 543 — Diameter of Binary Tree
4. 257 — Binary Tree Paths
5. 113 — Path Sum II
6. 105 — Construct Binary Tree from Preorder and Inorder Traversal
7. 106 — Construct Binary Tree from Inorder and Postorder Traversal
8. 199 — Binary Tree Right Side View
9. 103 — Binary Tree Zigzag Level Order Traversal
10. 236 — Lowest Common Ancestor of a Binary Tree
11. 98 — Validate Binary Search Tree
12. 230 — Kth Smallest Element in a BST

Goal: recognize postorder, backtracking, BFS variations, bounds, and tree construction.

55. Advanced: solve less obvious patterns

1. 124 — Binary Tree Maximum Path Sum
2. 437 — Path Sum III
3. 863 — All Nodes Distance K in Binary Tree
4. 297 — Serialize and Deserialize Binary Tree
5. 114 — Flatten Binary Tree to Linked List
6. 105 — Revisit construction and optimize index lookup
7. 99 — Recover Binary Search Tree
8. 968 — Binary Tree Cameras
9. 1372 — Longest ZigZag Path in a Binary Tree
10. 834 — Sum of Distances in Tree (tree DP; general tree)

Goal: combine multiple states, global answers, parent maps, graph traversal, and tree DP.

Final Checklist — What to Remember

1. A binary tree has at most two children; a BST has an ordering rule.
2. Preorder = Node, Left, Right.
3. Inorder = Left, Node, Right.
4. Postorder = Left, Right, Node.
5. DFS uses recursion or a stack; BFS uses a queue.
6. For level order, capture the queue size before processing a level.
7. Most recursive tree problems ask for information from children or pass information from ancestors.
8. Backtracking means choose, explore, undo.
9. A global answer is useful when the best answer can be anywhere in the tree.
10. A parent return value must represent a valid single path or contribution.
11. BST inorder is sorted; BST validation needs ancestor bounds.
12. Distance-K problems need a way to move to parents.
13. Always test null, one-node, skewed, duplicate, and negative-value cases as relevant.

The most important habit: Before writing code, explain in one sentence what your recursive helper means. If that sentence is precise, the base case, recursive calls, and return statement become much easier to derive.